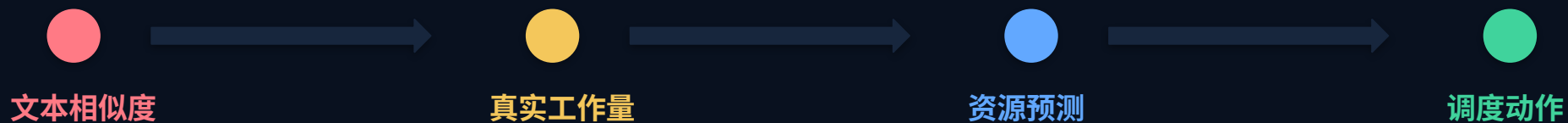


从“猜工具多久” 到“让 CPU 与 GPU 协同”

一条由失败结果推动的研究路线：预测什么、为什么失败、怎样变成真正有用的动作



Agent 不是一直在用 GPU：它在“思考”和“做事”之间切换

一个任务的真实时间线

GPU：生成计划

CPU：跑测试

GPU：读结果

CPU：改代码

GPU：继续推理

时间 →

机会

工具运行时，GPU 可能空出来，可以服务别的任务。

风险

工具突然结束，多批请求一起返回，首字延迟可能恶化。

核心问题

怎样知道“还有多少工作”，并把这信息变成安全动作？

这一阶段学到

研究对象应是完整任务的交替阶段，而不是孤立的一条 LLM 请求或 shell 命令。

我们先补齐了公开数据缺少的东西：原始命令与子命令级资源数据

SQLGlott • 200 tasks

同一仓库、任务相似
适合学习重复命令与测试阶段

PennyLane • 76 tasks

同一仓库、负载更高
含有效子命令级内核计数

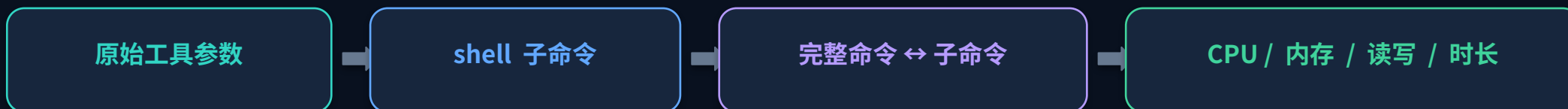
SWE • 377 tasks

跨 205+ 仓库
适合验证广度，不适合假装同仓库复用

TB • 239 tasks

工具更杂
适合看分布，不等于高并发实机

每条执行命令保留什么？



这一阶段学到

这批 trace 的价值不是“更多行”，而是能把一句 shell 命令拆成可解释的真实工作。

阶段 1：CacheWise 的整段参数相似度，在我们的 SWE 上反而更差

CacheWise 的直觉

把完整工具参数转成文本向量 → 聚类 → 用同类历史估计工具还要多久

为什么失败？

“python -m pytest” 说明要跑测试，却不知道仓库规模、依赖是否缓存、测试是否会下载数据。文字像，不等于剩余工作像。

我们的结果：越低越好



+0.587 s

C100 额外后悔

95% CI +0.120~+1.188

46 / 44

有帮助 / 有伤害

变化次数几乎对半

35.5 s

最差 1% 后悔

工具名对照 17.3 s

这一阶段学到

文本只能描述“在做什么”；资源预测还必须描述“做多少、做到哪、环境还差什么”。

阶段 2：同一仓库更可学，收益来自任务结构

SQLGlots50 · 四目标平均准确率（越高越好）



迁移到 PennyLane （越高越好）



任务感知预测增加了什么？

- 完整命令历史，而不是只看单个子命令
- 同一测试套件的第几次运行（环境可能已变热）
- pip / pytest 的等价工作表示；没有证据时回退子命令历史库



迁移收益来自多个 task，
但绝对提升仍然有限。

这一阶段学到 同仓库 trace 提供可重复结构；它能改进预测，却没有自动回答“调度该做什么”。

阶段 3：换索引结构不是答案，证据才是

评分单位始终是完整命令



子命令的作用

把复合命令拆细，找到是哪一段产生 CPU、内存、读写和时长。

历史库的作用

保存已完成任务的因果历史；当前任务和失败任务不能偷看。

预测器的作用

把多层证据合成完整命令的档位概率；没有可靠证据时回退。

这一阶段学到

换数据结构只有在它改变了可用证据、预测和下游动作时才有科研意义。

阶段 4：因素之间的联系，才能解释真实工作量

pip：收益上界很小

即使知道完美的 pip 工作类型，四目标平均准确率最多只提高 0.392 个百分点。

继续雕刻 package 规则不划算。

pytest：两个因素要一起看

只看并行进程数，高内存档召回率 2.381%。

同时看并行进程数 × 测试范围后，达到 54.762%。

通用 agent：没有捷径

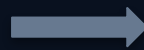
让 agent 从 trace 自动生成复杂模式，没有得到可区分的新状态；任意代码还带来泄漏与验证成本。

pytest 的关键不是识别名字，而是估算并行度与工作范围的乘法效应

并行进程数

×

测试范围



内存压力

这一阶段学到

离线 LM 可以抽取少量、可验证的因素；真正的成本仍应由 trace 学习，而不是让 LM 直接猜。

阶段 5：持续观察能提高准确率，但通常发现得太晚

SQLGlott：时长加权准确率（越高越好）

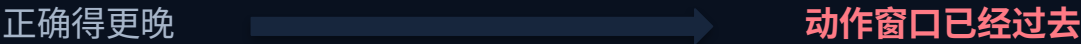
但它提前多久变得稳定正确？



49.2 ms
完整命令提前量

38.2 ms
子命令提前量

调度动作至少需要 500 ms 提前量



这一阶段学到 预测是否有价值，要同时看“正确多少”和“提前多久”；晚到的正确答案不能指导调度。

阶段 6：预测准确 $\neq$ 调度安全；运行时反馈反而更可靠

-45.010%

CPU 预留量

因果运行时反馈；259 tasks

+1.543%

工具自身变慢

反馈动作的代价

-14.620%

理想 CPU 调度

知道未来的上界

-48.823%

理想内存调度

时间变化的内存上界

为什么真实预测器没直接变成安全调度器？

- 峰值 CPU 不是可持续配额：按峰值太保守，按均值又会重叠爆发
- 内存三档不等于安全上界；少量低估就可能制造容量暴露
- 某些命令没有可用预测时，硬性资源准入会让全部任务互相等待

正确的研究顺序

1. 先证明动作有上界
2. 再设计不会死锁的反馈控制
3. 最后问预测器是否增加收益

这一阶段学到

调度器需要的是随时间变化的资源轨迹和安全退路，而不是把分类标签当作硬配额。

阶段 7：工具空档确实能换来约 33% 完成时间收益，但返回尾部变差

动作：前台任务进入长工具阶段时，临时放入一个等待任务



静态预测没触发：最高时长档从 30 s 开始，但动作需要 >34.7 s 才值得。

这一阶段学到

空档借用是有效动作；下一步不是再预测时长，而是保护前台任务返回时的 GPU 服务。

阶段 8：最大的机会来自 CPU 与 GPU 阶段错开，而不是只调一个资源

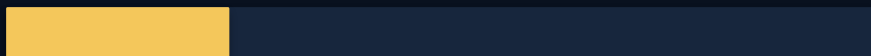
PennyLane 70 tasks • 理想上界 • 平均完成时间（越低越好）

只按 GPU 限制



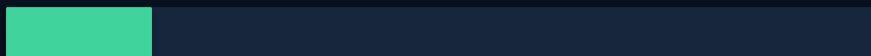
79,829 s

只按工具限制



21,705 s

联合看阶段



14,100 s

不是把并发固定得更大

联合上界最多同时放入 11 个任务，但每个资源仍不超容量。

而是让不同阶段互补

GPU 忙时少放推理；GPU 空时让 CPU 工具继续推进。

GPU 17.3%

CPU 37.7%

内存 14.2%

这些仍是 trace 回放上界，不是实机 GPU 结论。

这一阶段学到

真正的 frontier 是跨资源阶段协调：准入、优先级与状态保留必须一起看。

阶段 9：理想上界也没收益，方向才真正关闭

容器暂停 / 快照

即使假设暂停与恢复完全免费，平均完成时间仍恶化 2.664%。只释放 3.178% CPU 时间、1.316% 内存时间。

静态资源档位准入

更准的内存分类仍出现大量容量暴露；两种因果映射方式在 307 s 互相等待。

工具时长优先级

即使知道真实工具时长，短工具优先也只改善 0.435%；学出来只会更低。

CacheWise / C100

在当前模拟器中，预测只减少 1.481% 的 KV 重算；知道未来的上界也仅 9.620%。

通用 agent 规则生成

复杂输出没有创造新的可辨识状态，却增加泄漏、运行成本和验证成本。

本轮 vLLM 原生优先级

目前没有正式结果：前两组有效，第三组仅辅助 GPU 采样间隔超限。不能从中声称方法失败。

这一阶段学到

只有“理想上界也没收益”的方向才真正关闭；框架无效不能当作方法无效。

三个相关工作已覆盖很多“显而易见”的动作

Continuum

看见工具调用后，为 GPU 中间状态设置保留期限。

期限由工具历史、重算 / 重载成本和排队代价决定；到期就释放，避免无限占用。

已经覆盖：工具空档中的 KV 保留。

ThunderAgent

把完整 agent program 作为调度单位，区分推理 / 工具阶段。

按工具已运行时间降低其 KV 优先级，并做跨 GPU 恢复、迁移和工具环境准备。

已经覆盖：阶段感知 + 状态生命周期。

Agentix

截获完整程序的 LLM 调用，按已完成服务量或关键路径调度。

长请求偏向原模型副本保留局部性；短请求发到较空副本。

已经覆盖：程序级优先级 + 到达后的路由。

这一阶段学到

“看工具阶段” “保留 KV” “按 program 排队” “KV-local 路由” 都不能再作为我们的新贡献。

我们的可检验差异，只剩“工具内部进度能否让动作更好”

信息 / 动作	Continuum	ThunderAgent	Agentix	我们的位置
外层工具名 / 已运行时间	✓	✓	—	已有对照
程序级排队 / 到达后路由	先来先服务	✓	✓	已有对照
GPU 中间状态保留 / 迁移	保留期限	✓	局部性	已有对照
shell 子命令已完成到哪里	—	—	—	候选
CPU / 内存 / 读写的实时因果状态	—	—	—	候选
同时决定 GPU 返回与 CPU 工具执行	—	部分	—	候选

候选假设：比“工具已经跑了多久”更细的内部进度，能更早判断返回窗口与资源压力。

它们还覆盖了我们没有真正测过的系统场景

Continuum：更多模型、硬件与真实 agent

mini-SWE-Agent、BFCL Web、OpenHands；另跑 500 个真实 SWE 任务。采用任务到达流，覆盖 A100/H100/B200、单卡到 4 卡和分布式推理。

我们未覆盖：多模型、多硬件、真实任务到达与 500 个任务的规模。

ThunderAgent：高并发、多节点与强化学习采样

coding、routing、science agents 与强化学习采样；最高 96 个并发程序，模型跑在 H100 集群，Docker 工具环境放在独立 CPU 集群，并测环境准备与回收。

我们未覆盖：多节点、超大模型、强化学习与高并发。

Agentix：混合程序、并行搜索与多副本

ShareGPT 对话、BFCL 工具调用、LATS 并行搜索及其混合负载；按任务而非请求生成到达流，使用 1/4/8 张 A100，并测 2-4 个模型副本。

我们未覆盖：并程序图、混合负载与多副本局部性。

我们：工具内部更真实，系统规模更小

SQLGlue/PennyLane 保留真实命令、子命令与 CPU/内存/读写计数；SWE/TB 提供广度。但物理 GPU 证据只有单 A100、8 个固定任务的回放。

我们未覆盖：任务到达流、多副本、多 GPU、强化学习和混合 agent。

“更真实”不是一个排名，要看我们想证明什么

证明工具理解

最真实：同一大型仓库中的新任务，真实执行测试 / 安装命令，并刻意覆盖冷缓存、热缓存和不同测试范围。我们更接近，但环境对照还不够。

证明 GPU 服务

最真实：不同 agent 混合到达、从低负载扫到饱和、多个模型副本，并报告平均与最慢 1% 延迟。三篇相关工作明显更完整。

证明 CPU/GPU 协同

最真实：多个 coding agents 共享模型服务，在独立 CPU 节点执行真实工具；全部等待、变慢和状态移动都计入。ThunderAgent 最接近，但不看 shell 内部。

因此不能只选一种数据分布；需要两层互补验证

A · 方法是否有效

新鲜的同仓库任务 + 真实环境状态
回答：细粒度状态能不能改进动作？



B · 系统是否实用

同仓库与跨仓库混合 + 批量突发与到达流
回答：在真实竞争下是否仍改善完成时间和尾部？

这一阶段学到

下一套实验应保留我们的真实工具粒度，同时补上相关工作的并发、到达流和多副本场景。

下一步先做一个最小因果实验，不先造新系统

同一批真实 trace，比较三层信息是否真的改变动作



先在现有 trace 上做

问 C 是否比 B 改变了借入、返回保护或 KV 保留决策；不需要 GPU。

只有过门槛才上实机

至少改变足够多独立任务的动作，并在 trace 回放中同时改善完成时间与返回重叠。

实机对照不能缺

Continuum 式保留期限、ThunderAgent 式时间衰减、Agentix 式到达后优先级。

这一阶段学到

贡献不是“我们看得更细”，而是这份细粒度状态在相同成本下改变了更好的动作。

如果四个环节都成立，论文故事才完整



可能贡献 A

普通黑盒 agent trace 也能得到完整命令 / 子命令级资源状态。

可能贡献 B

细粒度状态改善跨 CPU/GPU 的实际调度动作。

可能贡献 C

在真实 agent 负载上同时改善完成时间与尾部。

这一阶段学到

单独的预测准确率不够；系统贡献必须落在完整任务完成时间、尾部与资源成本上。

带走四句话，然后继续问一个更尖锐的问题

- 01 —— 文本相似，不代表真实工作相似。
- 02 —— 同仓库结构能提高预测，但不自动产生调度收益。
- 03 —— 反馈和阶段错开已有强信号；硬档位准入容易不安全。
- 04 —— Continuum、ThunderAgent、Agentix 已覆盖外层阶段与 KV 动作。

下一问：工具内部进度是否能比“已运行多久”更早、更准地保护 GPU 返回，同时推进 CPU 工作？